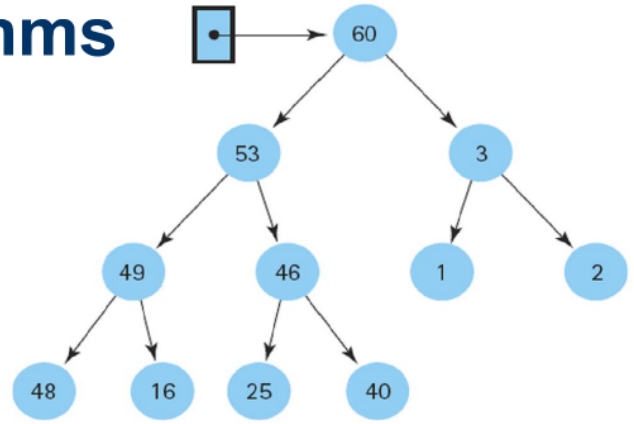
Data Structures & Algorithms



Trees

- tree is hierarchical (or non-linear) data structure.
- وهي هياكل البيانات المستخدمة لتمثيل هياكل الملفات الهرمية. بحيث ان الاوراق leaves هي ملفات files والعناصر الأخرى فوق الأوراق هي ادلة directories

Example: UNIX Directory

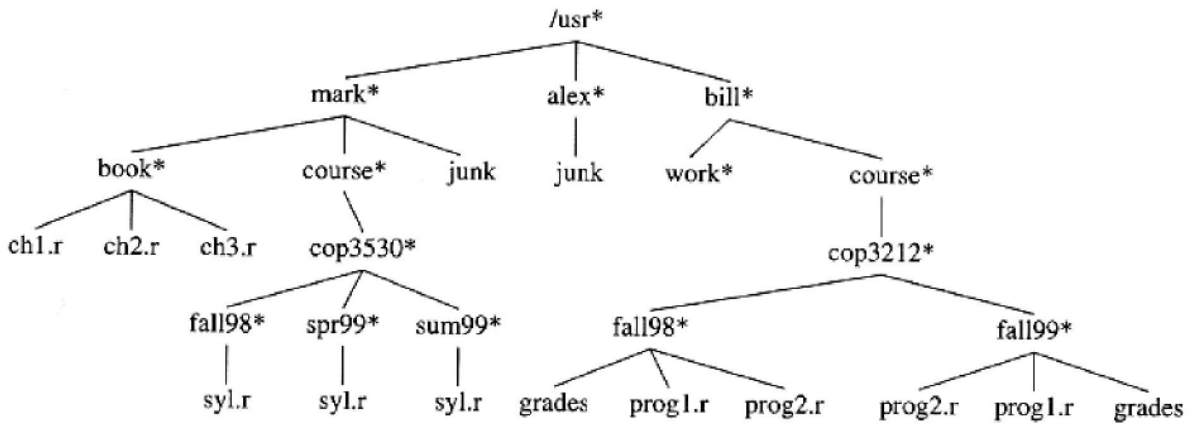


Figure 4.5 UNIX directory

Trees

- A tree is a collection of nodes
  - The collection can be empty
  - (recursive definition) If not empty, a tree consists of a distinguished node r (the *root*), and zero or more nonempty *subtrees* T_1, T_2, ..., T_k, each of whose roots are connected by a directed *edge* from r

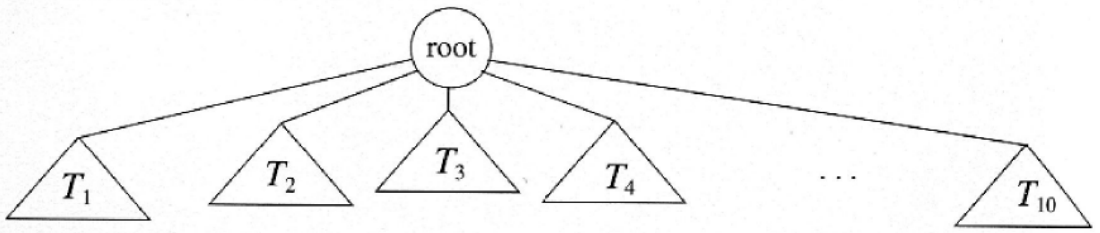


Figure 4.1 Generic tree

بعض المصطلحات Some Terminologies

- *Child* and *parent* الاطفال والاهل
  - Every node except باستثناء the root has one parent
  - A node can have an arbitrary اعتباطا number of children
- *Leaves* الاوراق
  - Nodes with no children
- *Sibling* الاشقاء
  - nodes with same parent
- *Path* الطريق
- *Edge* الخطوة
- *Length*
  - *number of edges on the path* عدد الخطوات في الطريق

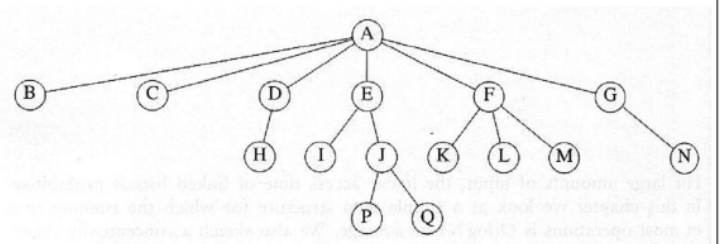


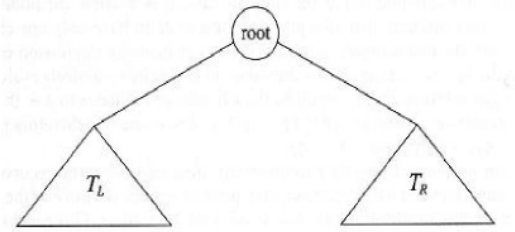
Figure 4.2 A tree

Some Terminologies

- ***Depth of a node*** عمق النود
  - **length of the unique path from the root to that node**
    طول الطريق من الجذر لتلك النود
  - **The depth of a tree is equal to the depth of the deepest leaf**
    عمق الشجرة مساوي لعمق ابعد ورقة
- ***Height* of a node** ارتفاع النود
  - **length of the longest path from that node to a leaf**
    هو طول اطول طريق من تلك النود الى الورقة
  - **all leaves are at height 0**
  - **The height of a tree is equal to the height of the root** ارتفاع الشجرة مساوي لارتفاع الجذر

Binary Trees الاشجار الثنائية

- A tree in which no node can have more than two children هي الاشجار والتي لاتحتوي أي نود فيها على اكثر من طفلين



Example: Expression Trees
شجرة التعابير

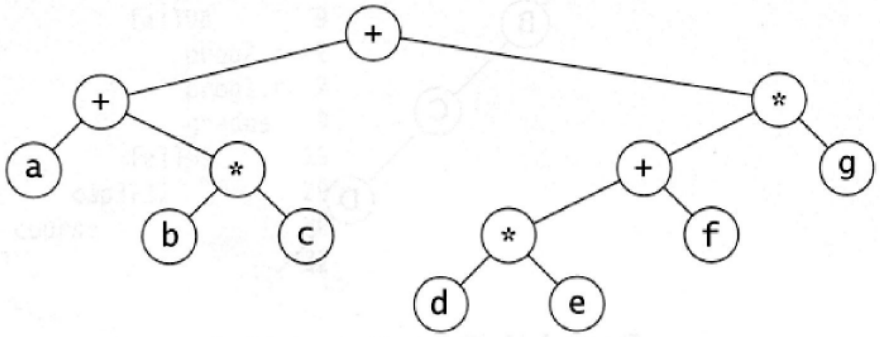


Figure 4.14 Expression tree for (a + b * c) + ((d * e + f) * g)

- Leaves are operands (constants or variables) الاوراق هي عوامل (ثوابت او متغيرات)
- The other nodes (internal nodes) contain operators باقي النود (النود الداخلية) تحتوي على المعاملات
- Will not be a binary tree if some operators are not binary لايمكن ان تكون الشجرة ثنائية اذا كان بعض المعاملات غير ثنائية

العمليات على الاشجار الثنائية :

1. Traversal tree
2. Search for node
3. Insert node
4. Delete node
5. Find min \ find max

1. Tree traversal استعراض الاشجار

- Used to print out the data in a tree in a certain order

يستخدم الاستعراض لطباعة البيانات في الشجرة بترتيب محدد مسبقا

Three types :

- Pre-order traversal
- Post-order traversal
- In-order traversal

Preorde:

- Preorder traversal
  - node, left, right
  - prefix expression
    - ++a*bc*+*defg

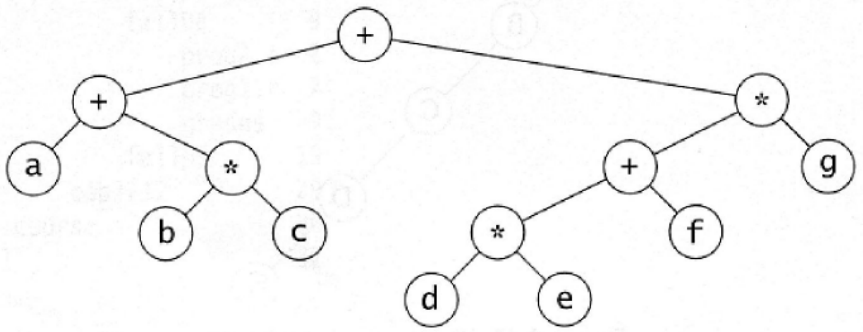


Figure 4.14 Expression tree for (a + b * c) + ((d * e + f) * g)

Postorder and Inorder

- Postorder traversal
  - left, right, node
  - postfix expression
    - abc*+de*f+g*+
- Inorder traversal
  - left, node, right.
  - infix expression
    - a+b*c+d*e+f*g

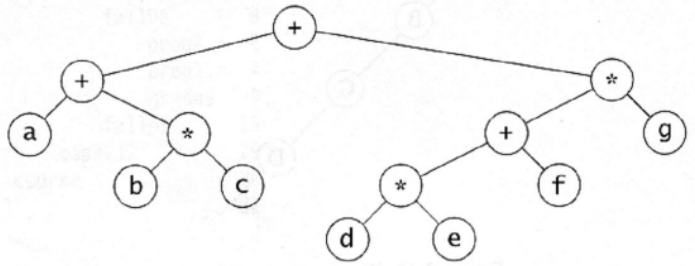
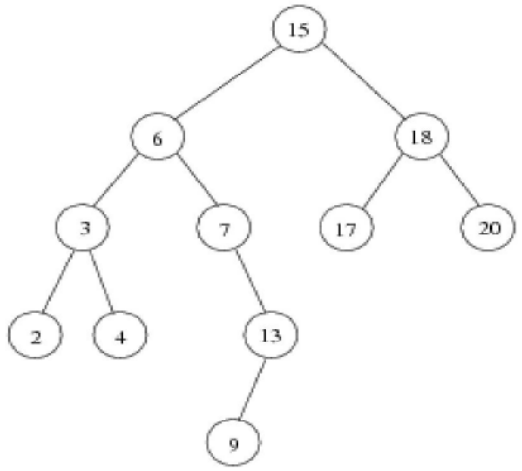


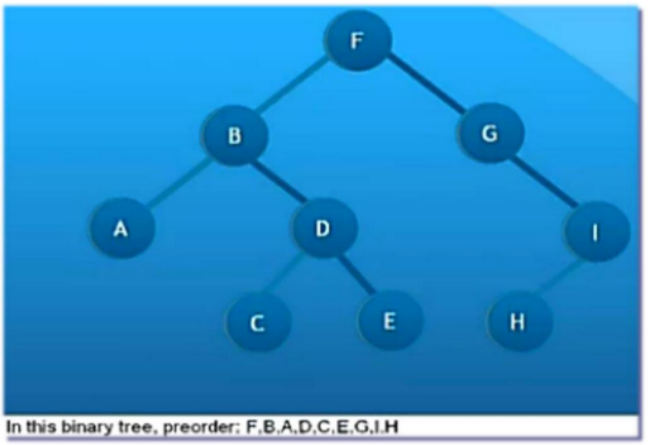
Figure 4.14 Expression tree for (a + b * c) + ((d * e + f) * g)

Example:
Print out all the keys in three traversal type:



Inorder: 2, 3, 4, 6, 7, 9, 13, 15, 17, 18, 20
Preorder : ?
Postorder : ?

Example:
Print out all the keys in three traversal type:



Post oreder : ?
In oreder : ?

Preorder :

```
/usr
    mark
        book
            ch1.r
            ch2.r
            ch3.r
        course
            cop3530
                fall98
                    syl.r
                spr99
                    syl.r
                sum99
                    syl.r
        junk
    alex
        junk
    bill
        work
        course
            cop3212
                fall98
                    grades
                    prog1.r
                    prog2.r
                fall99
                    prog2.r
                    prog1.r
                    grades
```

- Postorder

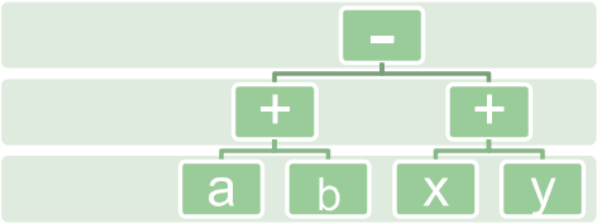
```
            ch1.r          3
            ch2.r          2
            ch3.r          4
        book              10
                syl.r      1
            fall98         2
                syl.r      5
            spr99          6
                syl.r      2
            sum99          3
        cop3530           12
    course                13
    junk                   6
mark                      30
    junk                   8
alex                       9
    work                   1
                grades     3
                prog1.r    4
                prog2.r    1
            fall98         9
                prog2.r    2
                prog1.r    7
                grades     9
            fall99        19
        cop3212           29
    course                30
bill                      32
/usr                      72
```

Example:
Print out all the keys in three traversal type:

Post = a b + x y + -

Pre = ?

In = ?



Binary Trees الاشجار الثنائية

- Implementation التمثيل
  - Because a binary tree has at most two children, we can keep direct pointers to them

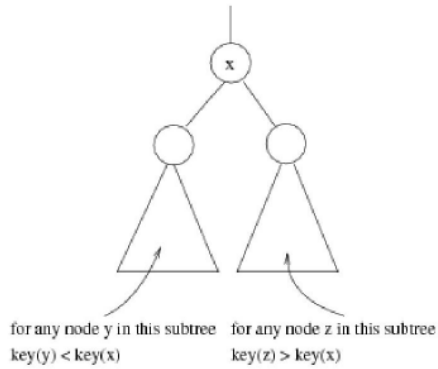
```
struct BinaryNode
{
    Object      element;      // The data in the node
    BinaryNode *left;         // Left child
    BinaryNode *right;        // Right child
};
```

Binary Search Trees

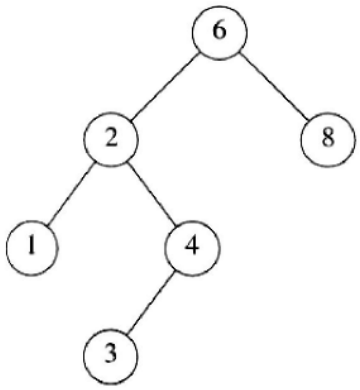
- Stores keys in the nodes in a way so that searching, insertion and deletion can be done efficiently.

Binary search tree propertyميزة

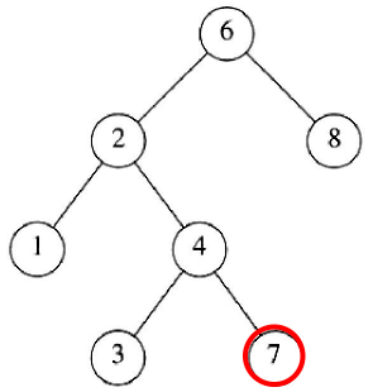
- For every node X, all the keys in its left subtree are smaller than the key value in X, and all the keys in its right subtree are larger than the key value in X
- جميع المفاتيح في الفروع Xلكل نود في الشجرة وكل الفاتيح في Xاليسرى اقل من قيم المفاتيح في Xالفروع اليمنى هي اكبر من قيمة المفتاح في النود



Binary Search Trees



A binary search tree



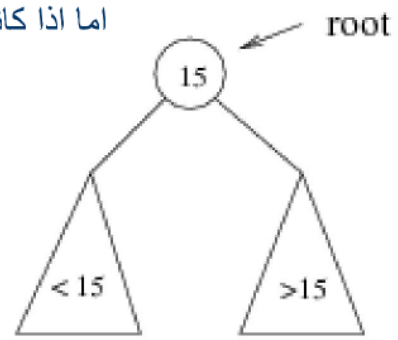
Not a binary search tree
Why?

2- البحث في الاشجار الثنائية Searching BST

- If we are searching for 15, then we are done.

لو اننا نبحث عن العنصر ١٥

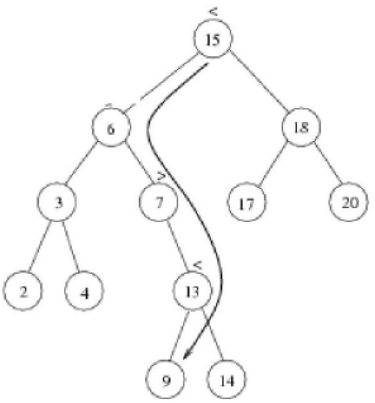
- If we are searching for a key < 15, then we should search in the left subtree. اذا كانت القيمة اقل من ١٥ نبحث في الجزء الايسر
- If we are searching for a key > 15, then we should search in the right subtree. اما اذا كانت القيمة اكب من ١٥ نبحث في الجزء الايمن



Example :

Search for 9:

1. compare 9:15(the root), go to left subtree;
2. compare 9:6, go to right subtree;
3. compare 9:7, go to right subtree;
4. compare 9:13, go to left subtree;
5. compare 9:9, found it!



Searching (Find)

- Find X: return a pointer to the node that has key X, or NULL if there is no such node

```
template <class Comparable>
BinaryNode<Comparable> *
BinarySearchTree<Comparable>::
find( const Comparable & x, BinaryNode<Comparable> *t ) const
{
    if( t == NULL )
        return NULL;
    else if( x < t->element )
        return find( x, t->left );
    else if( t->element < x )
        return find( x, t->right );
    else
        return t;    // Match
}
```

- Time complexity
  - O(height of the tree)

3- findMin/ findMax
ايجاد الاصغر وايجاد الاكبر

- Return the node containing the smallest element in the tree
  ارجاع قيمة النود التي تحتوي على اصغر قيمة
- Start at the root and go left as long as there is a left child.

The stopping point is the smallest element

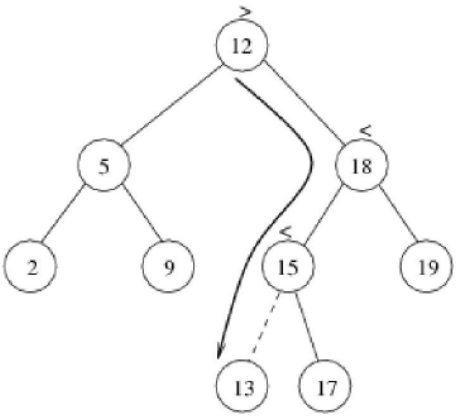
نبدا من الجذر ثم نذهب للفرع الايسر وهكذا نكرر حتى نصل لاخر نود تكون هي اصغر نود

```
template <class Comparable>
BinaryNode<Comparable> *
BinarySearchTree<Comparable>::findMin( BinaryNode<Comparable> *t ) const
{
    if( t == NULL )
        return NULL;
    if( t->left == NULL )
        return t;
    return findMin( t->left );
}
```

- Similarly for findMax نفس الشي بالنسبة لايجاد الاكبر
- Time complexity = O(height of the tree)

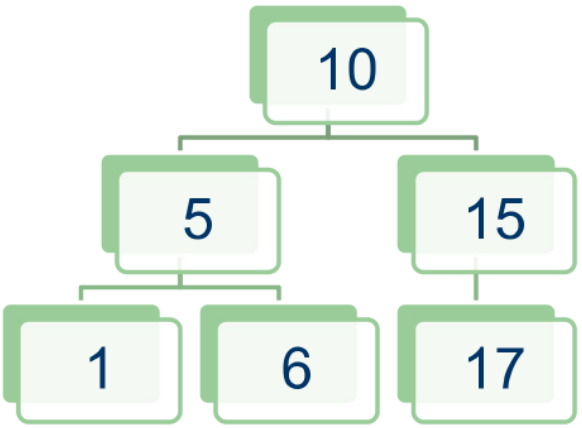
4- Insert الاضافة

- Proceed down the tree as you would with a find
- If X is found, do nothing (or update something)
- Otherwise, insert X at the last spot on the path traversed



- Time complexity = O(height of the tree)

Example:
insert key 14 into BST :



5- Delete

- When we delete a node, we need to consider how we take care of the children of the deleted node.

  عند الحذف لابد من الاخذ في الاعتبار اطفال النود المراد حذفه

  - This has to be done such that the property of the search tree is maintained.

  يطبق عادة عند البحث في الاشجار

Delete (cont)

Three cases: ثلاث حالات

(1) the node is a leaf عندما تكون النود ورقة

- Delete it immediately يتم الحذف مباشرة بدون مشاكل

(2) the node has one child عندما يكون للنود طفل

- Adjust a pointer from the parent to bypass that node لابد من تعديل المؤشر من اهل النود الى جانبي تلك النود (الطفل)

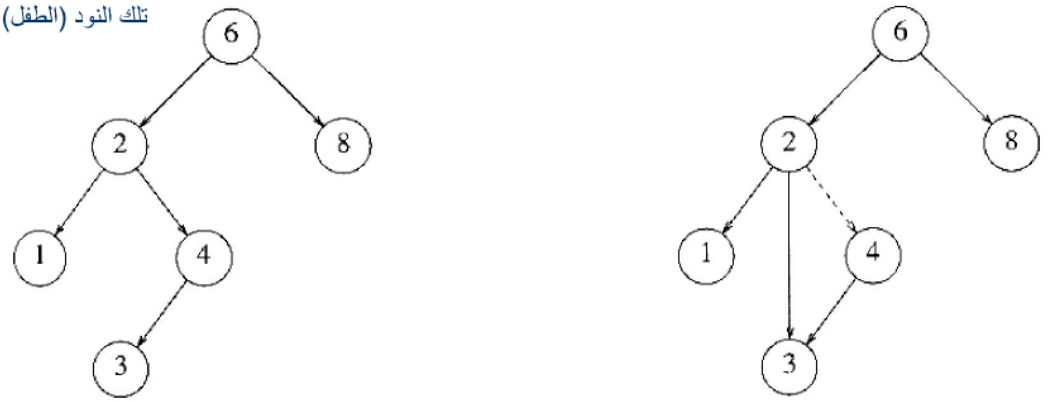


Figure 4.24 Deletion of a node (4) with one child, before and after

Delete (cont)

(3) the node has 2 children عندما يكون للنود طفلين

- replace the key of that node with the minimum element at the right subtree استبدال مفتاح النود مع اصغر عنصر في الفرع الايمن
- delete the minimum element حذف النود الاصغر
  - Has either no child or only right child because if it has a left child, that left child would be smaller and would have been chosen. So invoke case 1 or 2.

    اما لايكون له طفل (الحالة ١) او يكون له طفل ايمن فقط لان لو كان له طفل اصغر سيكون الطفل هو اصغر نود وسيتم اختياره

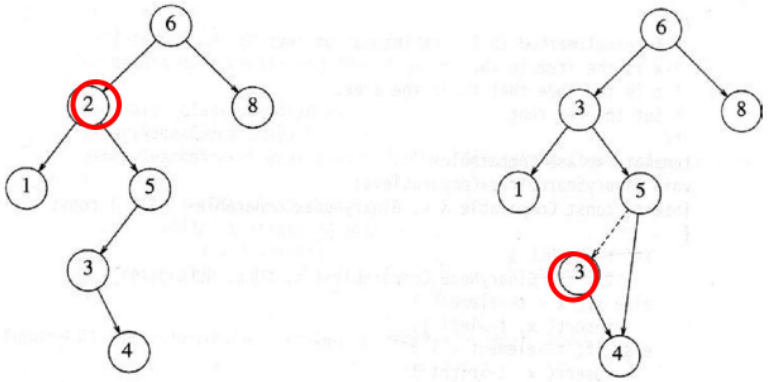
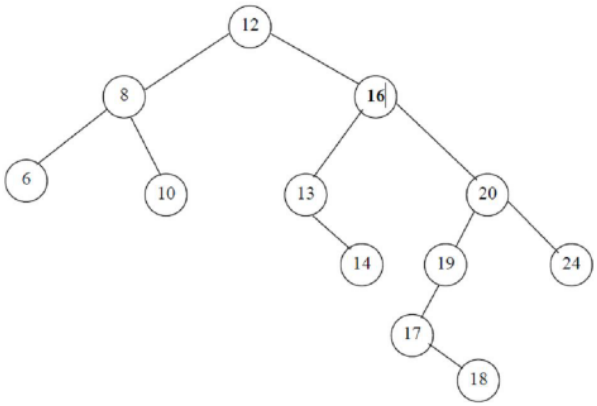


Figure 4.25 Deletion of a node (2) with two children, before and after

- Time complexity = O(height of the tree)

Example : Redraw the following trees after the node with key value 16 has been deleted.



End